

AUTOMOTIVE ENGINEERING AI

Standard Project Case Study Document

Five AI-enabled use cases for AUTOSAR, Functional Safety, Cybersecurity, Secure Code Review, and UDS Diagnostics

Reference Technology Position

Any suitable enterprise-approved or open-source LLM, local vector database such as FAISS or ChromaDB, Python backend using FastAPI or Flask, and a Streamlit-based engineering user interface.

Version 1.0 | September 2026

Prepared for internal project evaluation and solution planning

Document Control

Field	Details
Document Title	Automotive Engineering AI: Standard Project Case Study Document
Version	1.0
Document Type	Business and Technical Project Case Study
Purpose	Define business problems, proposed AI solutions, technology options, implementation approach, benefits, risks, and a common reference architecture.
Intended Audience	Engineering leadership, system architects, safety and cybersecurity teams, software developers, test teams, IT, data science, and program management.
Status	Reference / Proposal
Confidentiality	Internal use

Contents

- Executive Summary
- Common Project Vision and Design Principles
- Case Study 1: AUTOSAR HLD Document Analysis Assistant
- Case Study 2: Functional Safety HARA and Safety Requirement Management Assistant
- Case Study 3: Automotive Cybersecurity TARA Assistant
- Case Study 4: Secure Code Debugging and Review Assistant
- Case Study 5: UDS Diagnostics and Automated Test Generation Assistant
- Common Reference Solution Architecture
- Recommended Delivery Roadmap
- Conclusion

Executive Summary

Automotive engineering teams work with large volumes of architecture, safety, cybersecurity, source-code, diagnostic, and test information. Much of this information is stored in unstructured documents and disconnected tools. Manual interpretation creates delays, inconsistent outcomes, limited traceability, and strong dependency on experienced specialists.

This document defines five standard project case studies for an Automotive Engineering AI Platform. Each project combines a suitable Large Language Model with Retrieval-Augmented Generation, locally stored vector knowledge, Python-based application services, and a simple engineering web interface. The platform is intended to assist, not replace, responsible engineering decision-making.

No.	Project Case Study	Primary Value	Recommended Priority
1	AUTOSAR HLD Document Analysis	Faster architecture understanding and reusable design knowledge	Pilot
2	Functional Safety HARA and Requirements	Consistent analysis and end-to-end safety	Phase 2

No.	Project Case Study	Primary Value	Recommended Priority
		traceability	
3	Automotive Cybersecurity TARA	Improved threat coverage and reusable mitigation knowledge	Phase 2
4	Secure Code Debugging and Review	Higher developer productivity with IP protection	Phase 4
5	UDS Diagnostics and Test Generation	Faster, more consistent diagnostic test design	Phase 3

Common Project Vision and Design Principles

- Use any suitable open-source or enterprise-approved LLM based on benchmark results, security, licensing, and infrastructure constraints.
- Keep engineering documents, embeddings, prompts, responses, and source code within approved organizational boundaries.
- Use Retrieval-Augmented Generation so outputs are grounded in approved project documents.
- Store embeddings locally using FAISS or ChromaDB for pilots, with Milvus or self-hosted Weaviate as scale options.
- Use FastAPI as the preferred service layer, Flask as a lightweight alternative, and Streamlit for rapid engineering interfaces.
- Require citations, confidence indicators, audit logs, version control, and human approval for consequential outputs.
- Separate project collections and enforce role-based access to prevent cross-program data exposure.

Case Study 1

AUTOSAR HLD Document Analysis Assistant

Project Overview

Automate the extraction, organization, search, and validation of architectural knowledge from large AUTOSAR High-Level Design documents.

1. Business Context

High-Level Design documents describe software components, interfaces, ports, signals, dependencies, functional flows, and integration logic. They are often hundreds of pages long and maintained as unstructured PDFs. Architects, developers, integrators, and testers must manually interpret this content. The lack of structured extraction and search makes it difficult to identify mismatches, missing dependencies, inconsistent terminology, and incomplete test coverage.

2. Project Objective

- Reduce manual effort required to review HLD documents.
- Create a structured and searchable repository of architecture knowledge.
- Improve component, interface, signal, and dependency traceability.
- Support design reviews, onboarding, impact analysis, and test planning.

3. Scope

In Scope	Out of Scope
• PDF and supported document ingestion • OCR for scanned pages • Component, interface, port, signal, and flow extraction • Natural language search with citations • Document comparison and inconsistency reporting • Export of structured findings	• Automatic approval of architecture decisions • Modification of source HLD documents without review • Replacement of formal architecture governance

4. Primary Users and Stakeholders

- System architects
- AUTOSAR architects
- Software developers
- Integration engineers
- Test engineers
- Engineering managers

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

- Document upload and controlled ingestion
- Section, table, and metadata extraction
- Architecture entity recognition
- Interface and dependency mapping
- Natural language question answering
- Cited summaries and component reports
- Revision comparison
- Potential mismatch and completeness checks

7. High-Level Workflow

1. Upload or select an approved HLD document.
2. Extract text, tables, headings, and metadata.
3. Chunk content by section and architecture context.
4. Create embeddings and save them in the local vector database.
5. Retrieve relevant sections for each user query.
6. Generate grounded responses with page or section citations.
7. Allow engineer review, correction, and export.

8. Inputs and Outputs

Inputs	Expected Outputs
• AUTOSAR HLD documents • Interface specifications • Architecture descriptions • Component catalogues • Revision history	• Component and interface inventory • Dependency maps • Functional-flow summaries • Searchable Q&A responses • Potential inconsistency report • Structured export for downstream use

9. Suggested Technology Stack

Layer	Recommended Options	Deployment Note
User interface	Streamlit; Flask Web UI; optional enterprise portal	Streamlit for pilot, enterprise UI later
API and services	FastAPI or Flask	FastAPI preferred for modular APIs
LLM	Any approved LLM: Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent	Local, on-premises, or private environment

Layer	Recommended Options	Deployment Note
Embeddings	BGE, E5, Sentence Transformers, or equivalent	Evaluate with domain-specific queries
Vector database	FAISS or ChromaDB; Milvus or self-hosted Weaviate for scale	Persistent local storage and metadata filters
RAG orchestration	LangChain, LlamaIndex, or custom Python pipeline	Use controlled prompts and citations
Structured storage	SQLite or PostgreSQL	Projects, users, reviews, approvals, audit data
Deployment	Docker; optional Kubernetes	Network isolation and controlled access

10. Expected Business Benefits

- Reduced document-review effort
- Faster architecture validation
- Improved knowledge reuse
- Better onboarding
- Enhanced traceability to testing, safety, and cybersecurity

11. Indicative Success Metrics

Metric	Indicative Measurement
Document review effort	Percentage reduction against baseline manual review time
Extraction quality	Precision and recall for approved architecture entities
Answer groundedness	Percentage of responses with valid source citations
User adoption	Active users and repeat query volume
Issue discovery	Number of confirmed mismatches or missing dependencies identified

12. Key Risks and Controls

Risk	Recommended Control
Incorrect extraction from tables or diagrams	Use page-level citations and review queues
Hallucinated architecture relationships	Restrict answers to retrieved evidence and show confidence
Unauthorized access to project documents	Apply role-based access and project isolation
Outdated document versions	Enforce document version labels and approved-source status

13. Implementation Approach

Phase	Key Activities
1. Discovery	Collect representative HLDs, define taxonomy, questions, and success metrics.
2. Pilot	Build ingestion, local vector store, RAG search, Streamlit UI, and export.
3. Validation	Benchmark extraction and Q&A with architects and testers.
4. Scale	Add revision comparison, graph relationships, access control, and integrations.

Governance Principle

AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.

Case Study 2

Functional Safety HARA and Safety Requirement Management Assistant

Project Overview

Provide guided, evidence-based support for hazard identification, ASIL assessment, safety requirement drafting, and traceability management.

1. Business Context

Hazard Analysis and Risk Assessment under ISO 26262 depends on system architecture, operational scenarios, historical knowledge, and expert judgment. Manual spreadsheet-based execution can lead to inconsistent hazard identification, variation in severity, exposure, controllability assessments, and fragmented traceability from hazards to safety goals, requirements, architecture, and tests.

2. Project Objective

- Improve consistency of hazard identification and ASIL assessment.
- Reduce repetitive HARA preparation effort.
- Maintain continuous traceability across safety work products.
- Improve audit readiness while retaining expert approval.

3. Scope

In Scope	Out of Scope
• Item definition and functional input analysis • Hazard and hazardous-event suggestions • Guided S, E, and C assessment • Candidate ASIL recommendation with rationale • Safety goal and requirement drafting • Traceability and reporting	• Autonomous safety approval • Replacement of certified safety professionals • Use of unapproved standards content

4. Primary Users and Stakeholders

- Functional safety engineers
- System architects
- Requirement engineers
- Verification teams
- Safety managers
- Assessors and auditors

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

- Guided HARA workflow
- Retrieval of similar approved hazards
- Rule-based ASIL decision support
- Draft safety goals, FSRs, and TSRs
- Requirement quality checks
- Traceability matrix generation
- Decision history and audit evidence

7. High-Level Workflow

8. Ingest item definition, architecture, operational scenarios, and approved safety history.
9. Identify functions and malfunctioning behavior candidates.
10. Suggest hazards and hazardous events with source evidence.
11. Guide the engineer through severity, exposure, and controllability.
12. Apply approved decision rules and recommend an ASIL.
13. Draft safety goals and requirements.
14. Store approval decisions and traceability links.

8. Inputs and Outputs

Inputs	Expected Outputs
• Item definition • System architecture • Operational scenarios • Approved HARA libraries • Safety rules and templates • Verification artifacts	• Candidate hazards and hazardous events • ASIL recommendation and rationale • Draft safety goals, FSRs, and TSRs • Traceability matrix • Review and audit report

9. Suggested Technology Stack

Layer	Recommended Options	Deployment Note
User interface	Streamlit; Flask Web UI; optional enterprise portal	Streamlit for pilot, enterprise UI later
API and services	FastAPI or Flask	FastAPI preferred for modular APIs
LLM	Any approved LLM: Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent	Local, on-premises, or private environment

Layer	Recommended Options	Deployment Note
Embeddings	BGE, E5, Sentence Transformers, or equivalent	Evaluate with domain-specific queries
Vector database	FAISS or ChromaDB; Milvus or self-hosted Weaviate for scale	Persistent local storage and metadata filters
RAG orchestration	LangChain, LlamaIndex, or custom Python pipeline	Use controlled prompts and citations
Structured storage	SQLite or PostgreSQL	Projects, users, reviews, approvals, audit data
Deployment	Docker; optional Kubernetes	Network isolation and controlled access

10. Expected Business Benefits

- Reduced HARA cycle time
- More consistent risk classification
- Improved requirement quality
- Continuous safety traceability
- Lower audit preparation effort

11. Indicative Success Metrics

Metric	Indicative Measurement
HARA preparation time	Cycle-time reduction compared with prior projects
Consistency	Agreement rate across engineers for equivalent scenarios
Traceability completeness	Percentage of mandatory links completed
Recommendation acceptance	Accepted suggestions after expert review
Audit readiness	Time required to assemble review evidence

12. Key Risks and Controls

Risk	Recommended Control
Over-reliance on AI recommendations	Mandatory human approval and decision logging
Incorrect ASIL rationale	Combine deterministic rules with cited evidence
Unlicensed or uncontrolled standards content	Use only authorized internal knowledge sources
Traceability gaps after change	Run automated link-completeness and change-impact checks

13. Implementation Approach

Phase	Key Activities
1. Foundation	Define HARA data model, approved rules, templates, and governance.
2. Guided Pilot	Implement hazard suggestions, S/E/C forms, and local RAG.
3. Traceability	Add requirements, approval workflow, and graph relationships.
4. Validation	Conduct expert benchmarking, safety review, and controlled rollout.

Governance Principle

AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.

Case Study 3

Automotive Cybersecurity TARA Assistant

Project Overview

Assist cybersecurity engineers with architecture-aware asset identification, attack-surface analysis, threat scenarios, mitigations, and traceability.

1. Business Context

Threat Analysis and Risk Assessment under ISO/SAE 21434 requires engineers to interpret architecture, communication interfaces, data flows, assets, trust boundaries, and operational contexts. Manual execution is expertise-intensive, repeated across similar systems, and vulnerable to inconsistent threat coverage and mitigation selection.

2. Project Objective

- Improve asset and attack-surface coverage.
- Accelerate threat scenario generation and risk analysis.
- Standardize mitigation recommendations.
- Create a reusable, controlled cybersecurity knowledge base.

3. Scope

In Scope	Out of Scope
• Architecture and interface analysis • Asset and trust-boundary identification • Threat and damage scenario suggestions • Guided risk assessment • Mitigation and cybersecurity requirement recommendations • Traceability and reports	• Autonomous cybersecurity acceptance • Active vulnerability exploitation • Replacement of qualified cybersecurity review

4. Primary Users and Stakeholders

- Cybersecurity engineers
- System and network architects
- Security testers
- Software teams
- Cybersecurity managers
- Compliance and audit teams

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

- Asset extraction from architecture inputs
- CAN, Ethernet, diagnostics, OTA, and backend attack-surface mapping
- Threat scenario recommendation from approved libraries
- Risk assessment guidance
- Mitigation and requirement drafts
- Reusable threat library
- Traceability from assets to tests

7. High-Level Workflow

15. Ingest architecture, interfaces, data flows, and approved threat knowledge.
16. Identify candidate assets and trust boundaries.
17. Map exposed interfaces and attack surfaces.
18. Retrieve relevant threat patterns and prior scenarios.
19. Guide impact and attack-feasibility assessment.
20. Recommend cybersecurity goals, requirements, and mitigations.
21. Store decisions, rationale, and verification links.

8. Inputs and Outputs

Inputs	Expected Outputs
• System architecture • Communication matrix • Interface specifications • Asset catalogues • Approved threat libraries • Previous TARA artifacts	• Asset inventory • Attack-surface map • Threat and damage scenarios • Risk assessment support • Mitigation recommendations • Cybersecurity traceability report

9. Suggested Technology Stack

Layer	Recommended Options	Deployment Note
User interface	Streamlit; Flask Web UI; optional enterprise portal	Streamlit for pilot, enterprise UI later
API and services	FastAPI or Flask	FastAPI preferred for modular APIs
LLM	Any approved LLM: Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent	Local, on-premises, or private environment

Layer	Recommended Options	Deployment Note
Embeddings	BGE, E5, Sentence Transformers, or equivalent	Evaluate with domain-specific queries
Vector database	FAISS or ChromaDB; Milvus or self-hosted Weaviate for scale	Persistent local storage and metadata filters
RAG orchestration	LangChain, LlamaIndex, or custom Python pipeline	Use controlled prompts and citations
Structured storage	SQLite or PostgreSQL	Projects, users, reviews, approvals, audit data
Deployment	Docker; optional Kubernetes	Network isolation and controlled access

10. Expected Business Benefits

- Faster TARA execution
- Improved threat coverage
- Reduced dependence on limited expert capacity
- Consistent mitigation strategy
- Better audit and compliance support

11. Indicative Success Metrics

Metric	Indicative Measurement
Assessment cycle time	Time from architecture input to reviewed TARA
Asset coverage	Confirmed assets found versus expert baseline
Threat coverage	Relevant accepted threats per reviewed asset
Knowledge reuse	Percentage of scenarios sourced from approved reusable patterns
Traceability completeness	Assets, threats, controls, and tests linked

12. Key Risks and Controls

Risk	Recommended Control
Missed novel threats	Require expert red-team review and coverage checklist
Inappropriate mitigation recommendation	Show source, assumptions, and alternatives
Disclosure of sensitive architecture	Use isolated deployment and fine-grained access control
Stale threat knowledge	Apply ownership, review dates, and knowledge lifecycle controls

13. Implementation Approach

Phase	Key Activities
1. Taxonomy	Define asset, interface, threat, impact, and mitigation models.
2. Pilot	Build architecture ingestion, local retrieval, and Streamlit TARA workflow.
3. Risk Support	Add approved assessment rules and mitigation mapping.
4. Enterprise Scale	Integrate with requirements, testing, audit, and change management.

Governance Principle

AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.

Case Study 4

Secure Code Debugging and Review Assistant

Project Overview

Provide locally hosted AI assistance for code understanding, debugging, review, secure-coding checks, and structured findings without exposing proprietary source code.

1. Business Context

ECU software teams perform continuous debugging, peer review, MISRA-oriented analysis, security review, and performance investigation. Quality depends heavily on individual experience, while external AI tools may be prohibited due to source-code confidentiality, intellectual property, data retention, and regulatory concerns.

2. Project Objective

- Reduce debugging and review effort.
- Improve consistency of review quality.
- Ground recommendations in internal coding rules and approved guidance.
- Keep source code and model interactions inside organizational boundaries.

3. Scope

In Scope	Out of Scope
• Code explanation and module summaries • Compiler, static-analysis, and log interpretation • Candidate defect and root-cause suggestions • MISRA-oriented review assistance • Security-focused code review • Structured finding reports and repository integration	• Automatic code merge or release approval • Replacement of qualified reviewers or certified tools • External transmission of source code

4. Primary Users and Stakeholders

- Embedded software developers
- Code reviewers
- Technical leads
- Cybersecurity engineers
- CI/CD administrators
- Quality managers

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

- Repository-aware code retrieval
- Function, module, dependency, and control-flow summaries
- Analysis of compiler warnings and logs
- Potential null-pointer, boundary, resource, and state issues
- Coding-standard explanation
- Secure-coding recommendations
- Structured findings with evidence and status

7. High-Level Workflow

22. Authenticate the user and verify repository permissions.
23. Retrieve only authorized code and relevant internal guidelines.
24. Collect compiler, static-analysis, or runtime evidence.
25. Generate contextual findings and remediation suggestions.
26. Show file, function, source evidence, rule, and confidence.
27. Allow developer and reviewer disposition.
28. Write approved findings to the internal workflow system.

8. Inputs and Outputs

Inputs	Expected Outputs
• Source code • Build logs • Compiler warnings • Static-analysis reports • Internal coding standards • Historical approved findings	• Code summaries • Candidate root causes • Review findings • MISRA-oriented explanations • Security observations • Suggested fixes • Structured review report

9. Suggested Technology Stack

Layer	Recommended Options	Deployment Note
User interface	Streamlit; Flask Web UI; optional enterprise portal	Streamlit for pilot, enterprise UI later
API and services	FastAPI or Flask	FastAPI preferred for modular APIs
LLM	Any approved LLM: Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent	Local, on-premises, or private environment

Layer	Recommended Options	Deployment Note
Embeddings	BGE, E5, Sentence Transformers, or equivalent	Evaluate with domain-specific queries
Vector database	FAISS or ChromaDB; Milvus or self-hosted Weaviate for scale	Persistent local storage and metadata filters
RAG orchestration	LangChain, LlamaIndex, or custom Python pipeline	Use controlled prompts and citations
Structured storage	SQLite or PostgreSQL	Projects, users, reviews, approvals, audit data
Deployment	Docker; optional Kubernetes	Network isolation and controlled access

10. Expected Business Benefits

- Faster debugging
- More consistent reviews
- Improved knowledge transfer
- Reduced defect leakage
- Protection of source-code IP

11. Indicative Success Metrics

Metric	Indicative Measurement
Debug resolution time	Mean time to identify and resolve selected defect classes
Review throughput	Modules or change requests reviewed per engineer
Finding precision	Confirmed actionable findings versus false positives
Defect leakage	Review-related defects found in later test phases
Security compliance	Percentage of code processed within approved boundaries

12. Key Risks and Controls

Risk	Recommended Control
Exposure of proprietary code	Local inference, encryption, network isolation, and audit logs
False positives or unsafe fixes	Require compilation, test, and reviewer validation
Prompt injection through repository content	Treat repository text as untrusted and separate instructions from content
Excessive access to unrelated modules	Enforce repository and path-level authorization

13. Implementation Approach

Phase	Key Activities
1. Secure Sandbox	Deploy local model, access controls, and isolated pilot repositories.
2. Review Pilot	Add code Q&A, coding-guideline RAG, and structured findings.
3. Tool Integration	Connect static analysis, CI logs, and review workflow.
4. Controlled Rollout	Benchmark quality, monitor risks, and expand by repository.

Governance Principle

AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.

Case Study 5

UDS Diagnostics and Automated Test Generation Assistant

Project Overview

Help test engineers interpret approved diagnostic specifications, construct valid requests, generate test cases, and validate expected ECU responses.

1. Business Context

UDS testing requires engineers to interpret ISO 14229 concepts together with OEM-specific and ECU-specific implementations. They must manually construct requests, identify session and security prerequisites, determine expected positive and negative responses, and translate test intent into automation scripts. The volume of services, subfunctions, identifiers, routines, and variants increases inconsistency and error risk.

2. Project Objective

- Reduce diagnostic specification interpretation effort.
- Generate consistent positive and negative test scenarios.
- Validate request and expected-response structures.
- Accelerate conversion of requirements into executable test templates.

3. Scope

In Scope	Out of Scope
• Approved standard and OEM knowledge retrieval • UDS service and parameter guidance • Request-message construction • Positive and negative test generation • Expected-response validation • Export to Python, CANoe, CAPL, or approved frameworks	• Execution against production vehicles without controls • Use of unauthorized standards content • Unreviewed automatic acceptance of ECU behavior

4. Primary Users and Stakeholders

- Diagnostic test engineers
- ECU software developers
- Validation engineers
- Test automation developers
- Technical leads
- Quality managers

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

- Natural language diagnostic knowledge search
- Service, subfunction, DID, RID, NRC, session, and security guidance
- Request structure validation
- Test case generation with preconditions and pass/fail criteria
- Positive and negative coverage generation
- Automation template export
- Response comparison and reporting

7. High-Level Workflow

29. Ingest authorized UDS, OEM, ECU, and project specifications.
30. Extract services, parameters, preconditions, and response rules.
31. Store embedded chunks in project-specific local collections.
32. Accept a diagnostic requirement or engineer query.
33. Retrieve relevant evidence and apply deterministic validation rules.
34. Generate a structured test case and optional automation template.
35. Require engineer review before controlled execution.

8. Inputs and Outputs

Inputs	Expected Outputs
• Authorized UDS specifications • OEM diagnostic specifications • ECU diagnostic extract • Requirements • Existing tests • Automation templates	• Diagnostic guidance with citations • Validated request template • Expected positive response • Negative-response scenarios • Structured test case • Automation script template • Coverage report

9. Suggested Technology Stack

Layer	Recommended Options	Deployment Note
User interface	Streamlit; Flask Web UI; optional enterprise portal	Streamlit for pilot, enterprise UI later
API and services	FastAPI or Flask	FastAPI preferred for modular APIs
LLM	Any approved LLM: Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent	Local, on-premises, or private environment

Layer	Recommended Options	Deployment Note
Embeddings	BGE, E5, Sentence Transformers, or equivalent	Evaluate with domain-specific queries
Vector database	FAISS or ChromaDB; Milvus or self-hosted Weaviate for scale	Persistent local storage and metadata filters
RAG orchestration	LangChain, LlamaIndex, or custom Python pipeline	Use controlled prompts and citations
Structured storage	SQLite or PostgreSQL	Projects, users, reviews, approvals, audit data
Deployment	Docker; optional Kubernetes	Network isolation and controlled access

10. Expected Business Benefits

- Faster test design
- Improved positive and negative coverage
- Reduced request-construction errors
- Consistent test methodology
- Lower onboarding effort

11. Indicative Success Metrics

Metric	Indicative Measurement
Test design time	Average time to create reviewed diagnostic test cases
Generation accuracy	Accepted request and response structures
Coverage improvement	Positive, negative, session, security, and timing coverage
Reuse rate	Percentage of generated tests based on approved templates
Defect detection	Confirmed ECU issues found through generated cases

12. Key Risks and Controls

Risk	Recommended Control
Incorrect diagnostic message generation	Apply deterministic protocol rules and expert review
Misinterpretation of OEM-specific behavior	Separate standard, OEM, ECU, and project collections
Unsafe execution sequence	Execute only in approved test environments with safeguards
Standards licensing violation	Use authorized source documents and access controls

13. Implementation Approach

Phase	Key Activities
1. Knowledge Pilot	Ingest authorized documents and deliver cited diagnostic Q&A.
2. Test Generation	Add structured test templates and rule-based message validation.
3. Automation	Export to Python, CANoe, CAPL, or approved frameworks.
4. Scale	Add project isolation, coverage analytics, and toolchain integration.

Governance Principle

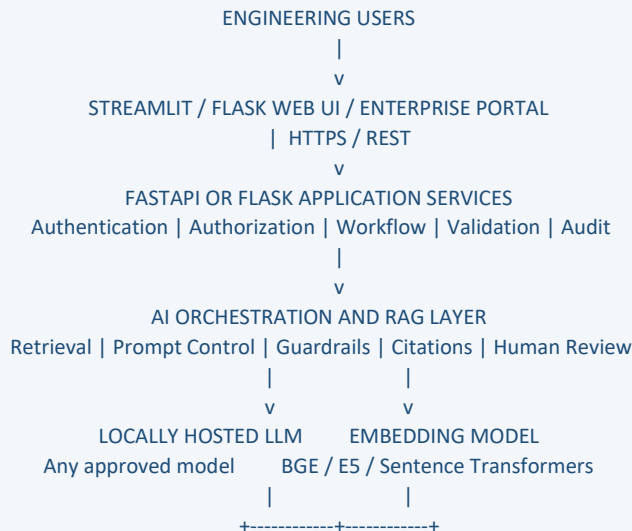
AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.

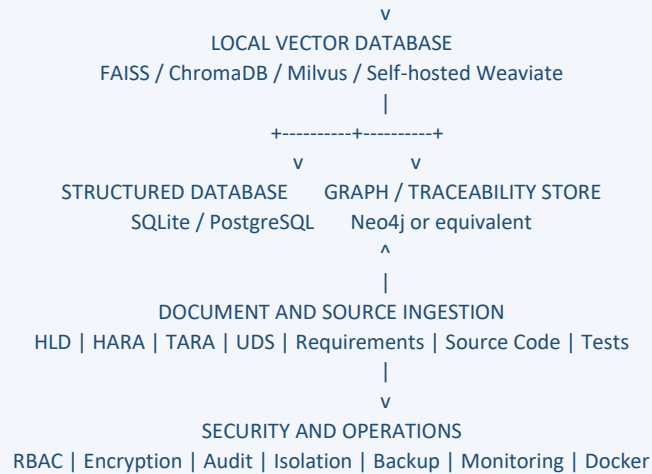
Common Reference Solution Architecture

The following reference architecture supports all five case studies. Components may be selected according to data classification, user scale, model performance, licensing, infrastructure, and organizational standards.

Architecture Layer	Purpose	Technology Options
User Interface	Chat, search, upload, review, forms, reports, dashboards	Streamlit; Flask Web UI; enterprise portal
Application and API	Authentication, workflow, business logic, validation, audit	FastAPI preferred; Flask alternative
AI Orchestration	Retrieval, prompt control, citations, response validation	LangChain; LlamaIndex; custom Python
LLM Inference	Reasoning, extraction, summarization, generation	Any approved LLM, locally or privately hosted
Embedding Service	Convert approved content into vectors	BGE; E5; Sentence Transformers; equivalent
Local Vector Store	Semantic retrieval with persistent project collections	FAISS; ChromaDB; Milvus; self-hosted Weaviate
Structured Store	Users, projects, reviews, approvals, audit data	SQLite for pilot; PostgreSQL for production
Graph Store	Cross-artifact relationships and traceability	Neo4j or equivalent
Ingestion	Parse documents, tables, source code, metadata, and OCR	PyMuPDF; PDFPlumber; PyPDF; Tesseract; custom parsers
Operations and Security	Deployment, monitoring, isolation, backup, access control	Docker; optional Kubernetes; enterprise IAM and logging

Logical Architecture Diagram





Reference Processing Flow

36. The user authenticates and selects an authorized project workspace.
37. The user uploads a document or selects an approved repository.
38. The backend validates file type, access rights, version, and project context.
39. The ingestion layer extracts text, tables, code, and metadata, using OCR where required.
40. Content is divided into context-aware chunks and converted into embeddings.
41. Embeddings and metadata are stored in a local, project-specific vector collection.
42. A user query is submitted through Streamlit or another approved web interface.
43. FastAPI or Flask retrieves relevant evidence from the vector database.
44. The approved LLM generates a response using only the supplied context and configured instructions.
45. The response includes source citations, confidence or limitation indicators, and review status.
46. Deterministic rules are applied where protocol, safety, risk, or validation logic requires them.
47. The engineer reviews, accepts, edits, or rejects the output.
48. Approved artifacts and decisions are stored in the structured database.
49. Traceability links are maintained in the graph store and included in audit reports.

Recommended Delivery Roadmap

Stage	Focus	Indicative Outcome
Stage 1: HLD Pilot	Document ingestion, local vector search, cited Q&A, component and interface extraction	Validated technical foundation and measurable productivity baseline
Stage 2: HARA and TARA	Rule-based support, expert workflows, controlled knowledge, graph traceability	Consistent guided analysis and improved audit readiness
Stage 3: UDS	Diagnostic Q&A, request validation, test generation, automation templates	Faster and more consistent diagnostic validation
Stage 4: Secure Code Review	Repository access control, code retrieval, static-analysis integration, local code model	Developer productivity with strong IP protection
Stage 5: Platform Scale	Enterprise IAM, observability, governance, integrations, model evaluation, operations	Reusable Automotive Engineering AI Platform

Conclusion

The five projects share a common foundation and can be delivered as independent modules or as an integrated Automotive Engineering AI Platform. A practical pilot uses an approved locally hosted LLM, BGE or E5 embeddings, ChromaDB or FAISS for persistent local retrieval, FastAPI for backend APIs, and Streamlit for the engineering interface. As adoption grows, PostgreSQL, Neo4j, Milvus or self-hosted Weaviate, enterprise identity, and container orchestration can be added.

The recommended starting point is the AUTOSAR HLD Document Analysis Assistant because it offers strong business value, comparatively lower decision risk, and a reusable architecture knowledge base for HARA, TARA, diagnostics, and testing. Across all modules, outputs must remain evidence-based, access-controlled, auditable, and subject to qualified human review.

Recommended Pilot Stack

Locally hosted approved LLM + BGE/E5 embeddings + ChromaDB or FAISS + FastAPI backend + Streamlit frontend + SQLite/PostgreSQL for workflow data + Docker deployment.